

Metaxy: Field-Level Metadata Management for Incremental Multimodal ML Pipelines

Daniel Gafni
Anam
United Kingdom
danielgafni16@gmail.com

Georg Heiler
Complexity Science Hub Vienna (CSH)
Austria
Austrian Supply Chain Intelligence Institute (ASCI)
Austria
heiler@csh.ac.at

ABSTRACT

Multimodal ML pipelines incur substantial GPU costs, yet existing orchestrators invalidate entire downstream datasets when any processing step changes—even for unaffected fields. We present **Metaxy**, an open-source Python library providing *field-level dependency tracking at record granularity*. Feature definitions form a directed acyclic graph; for each record, hierarchical version hashes propagate changes along field-level edges. A `resolve_update` operation returns exactly those records that are new, stale, or orphaned, enabling selective recomputation. In production across two organizations processing millions of multimodal samples, Metaxy eliminates redundant GPU work while preserving complete per-record lineage for reproducibility.

CCS CONCEPTS

• **Information systems** → **Data provenance**; • **Software and its engineering** → *Software libraries and repositories*;

KEYWORDS

metadata management, incremental processing, feature engineering, multimodal ML, data provenance, reproducibility

1 INTRODUCTION

Modern ML increasingly relies on multimodal data—video, audio, images, and text—processed through GPU-accelerated pipelines. Unlike tabular ETL, where re-execution is cheap, multimodal steps such as speech-to-text transcription or embedding generation cost orders of magnitude more per record. Data orchestrators like Dagster [1] model pipelines as graphs of *assets* (tables or datasets) and track freshness at the asset level: when a processing step changes, every downstream record is recomputed—even those whose inputs are unchanged.

Consider a video pipeline extracting both audio transcripts and face detections (Figure 1). Improving only the audio denoising algorithm triggers recomputation of face detections that depend solely on video frames—a costly false positive. This problem compounds as pipelines grow and is especially acute for iterative, agent-driven ML development, where rapid experimentation on feature pipelines demands that each iteration recomputes only what is strictly necessary.

We address this gap with **Metaxy**,¹ a standalone metadata layer operating at *field* and *record* granularity that enables selective recomputation—both cost-efficient and supportive of rapid experimentation.

¹<https://github.com/anam-org/metaxy>

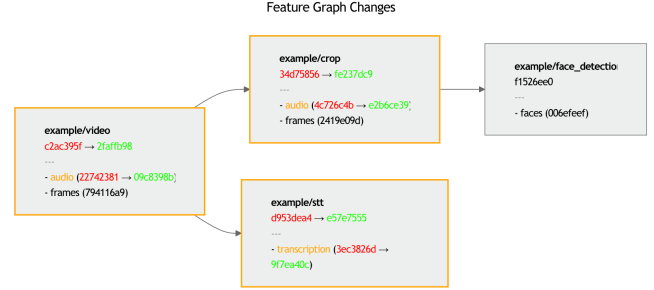


Figure 1: Feature graph after an audio algorithm change. Orange borders indicate affected features; gray borders remain unchanged. Red/green hashes show old/new field versions. The audio field change propagates to `example/crop` and `example/stt`, but `example/face_detection` (depending only on frames) is unaffected.

2 DESIGN

2.1 Feature Definitions and Dependency Graph

Feature definitions are registered through a common specification interface.² Each feature specifies identifier columns, computed fields with explicit code versions, and field-level dependencies on upstream features via a `FeatureSpec`. At initialization, Metaxy constructs a global DAG where nodes are feature fields and edges encode data flow. This graph enables topological version propagation: when an upstream field changes, only transitively dependent downstream fields are marked stale. The system handles one-to-one, one-to-many, and many-to-one lineage relationships, covering common multimodal patterns such as expanding a video into per-frame crops or aggregating audio segments into a transcript summary.

2.2 Hierarchical Versioning

Version computation operates through deterministic hashing at multiple levels: (1) *field code versions*—user-specified strings marking algorithmic changes; (2) *field provenance*—per-field hashes combining current code versions with upstream record field versions; (3) *record provenance*—a sample-level hash derived from the per-field provenance map. Metaxy also exposes user-controlled data

²The most common authoring pattern uses Python classes extending a Pydantic base model, but the same graph can also be populated from `metaxy.lock` or directly from the metadata store.

versions that default to provenance, allowing applications to encode direct data-version decisions when needed. When `resolve_update` is invoked, Metaxy compares expected provenance (computed from the current graph and upstream data) against stored provenance:

```
inc = store.resolve_update("video/speech_to_text")
# inc.new:    records not yet processed
# inc.stale:  records with changed expected provenance
# inc.orphaned: records removed upstream
to_process = pl.concat([inc.new, inc.stale])
```

The returned `Increment` contains exactly those records requiring work; unchanged records are skipped entirely.

2.3 Pluggable Storage, Orchestration, and Execution

Metaxy separates metadata from storage, orchestration, and execution. The `MetadataStore` abstraction supports multiple backends (e.g., DuckDB for local prototyping, ClickHouse or BigQuery or ducklake for production, or lakehouse formats like Delta Lake). Version computation is pushed into the store via SQL where possible, avoiding data transfers. Backend-agnostic dataframe interoperability is achieved through Narwhals [2] and Ibis [4].

For orchestration, Metaxy integrates with Dagster assets. The `metaxify` decorator enriches asset definitions with lineage metadata while leaving scheduling decisions to the orchestrator. For execution, Metaxy can be paired with Ray [7] via dedicated data-source and datasink classes. The orchestrator decides *which feature* to compute; Metaxy resolves *which samples actually require it*; the execution engine processes only the delta.

3 RELATED WORK

DVC [6] versions datasets at file level without tracking individual records. Asset orchestrators such as Dagster [1] schedule re-computation at asset granularity rather than tracking per-record, per-field provenance. DataChain [5] supports delta processing for multimodal workflows but couples its incremental model to its own framework. Metaxy fills this gap as a *standalone* metadata layer providing field-level tracking at record granularity, decoupled from any specific compute or storage backend.

4 PRODUCTION EXPERIENCE

Metaxy has been in production at Anam³, a conversational AI avatar company, since December 2025, processing millions of training samples for a real-time interactive avatar platform powered by a custom video generation model. The pipeline spans audio extraction, speech-to-text, face detection, video cropping, and embedding generation; the most expensive stages are GPU-accelerated. Before Metaxy, changing crop resolution triggered recomputation of all downstream steps, including audio-only stages unaffected by the change. With field-level tracking, only genuinely affected records are reprocessed. The append-only metadata design preserves complete per-record lineage, enabling retrospective audits and experiment reproducibility without additional bookkeeping.

At ASCII (Austrian Supply Chain Intelligence Institute), Metaxy powers incremental document processing for knowledge extraction from large-scale web crawls [3].

The same feature definitions can be reused across local prototyping and production deployments, reducing operational friction.

5 CONCLUSION

Metaxy introduces field-level dependency tracking at record granularity as a standalone metadata layer for multimodal ML pipelines. By computing precise diffs before execution, it eliminates redundant GPU work while preserving complete lineage. For the SAO community, Metaxy demonstrates that fine-grained provenance and cost-efficient incremental processing—properties essential for data systems supporting iterative and agentic ML workflows, feature pipeline iteration loops, and operational cost control—can be achieved through a lightweight layer that complements existing orchestrators rather than replacing them. The library is open-source at <https://github.com/anam-org/metaxy>.

REFERENCES

- [1] Dagster Labs. 2025. Dagster: Data Orchestration Platform. <https://dagster.io>. (2025).
- [2] Marco Gorelli and contributors. 2025. Narwhals: Write Once, Run on Many DataFrames. <https://narwhals.readthedocs.io>. (2025).
- [3] Georg Heiler, Hernan Picatto, Maximilian Heß, and Martin Pfister. 2025. dagster-slurm: Reproducible Data Orchestration on HPC. <https://github.com/ascii-supply-networks/dagster-slurm>. (2025).
- [4] Ibis Project. 2025. Ibis: Portable DataFrames for Different Backends. <https://ibis-project.org>. (2025).
- [5] Iterative, Inc. 2025. DataChain: AI Data Warehouse for Multimodal Data. <https://datachain.ai>. (2025).
- [6] Ruslan Kuprieiev, Dmitry Petrov, et al. 2025. DVC: Data Version Control. <https://dvc.org>. (2025).
- [7] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melanie Paul, Michael I. Jordan, and Ion Stoica. 2018. Ray: A Distributed Framework for Emerging AI Applications. In *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.

³<https://www.anam.ai>